TASKFLOW
TASK MANAGEMENT WEB APPLICATION

MINOR PROJECT REPORT

Submitted by
Nikhat Siddekee

Course
B.Tech in Computer Science and Engineering

Project Guide
Pratham Saxena

Academic Year
2026

CERTIFICATE

This is to certify that Nikhat Siddekee, a student of B.Tech in Computer Science and Engineering, has completed the minor project titled “TaskFlow: Task Management Web Application” under the guidance of Pratham Saxena.

This report describes the project work completed by the student.

Project Guide: Pratham Saxena
Student: Nikhat Siddekee
Date: 24 October 2026

ACKNOWLEDGEMENT

I would like to thank my project guide, Pratham Saxena, for the guidance and feedback provided during the development of this project.

I also thank everyone who supported me while planning, developing, and documenting TaskFlow.

ABSTRACT

TaskFlow is a browser-based task management application developed using HTML, CSS, and JavaScript. It helps users create and organize tasks, assign priorities and due dates, update task details, and track whether work is pending or complete.

The application includes a dashboard showing task totals, search, status and priority filters, sorting options, and visual highlighting for overdue pending tasks. It uses browser Local Storage to retain tasks after the page is refreshed.

This project demonstrates the use of front-end web technologies to build an interactive and responsive application.

1. INTRODUCTION

Managing assignments and everyday work can be difficult when tasks, deadlines, and priorities are not organized. A task management application can provide a clear place to record work and track its progress.

TaskFlow is a web application for managing tasks in a browser. Users can add task details, set a priority and due date, find tasks using search and filters, and mark tasks as complete or pending.

## 1.1 PROBLEM STATEMENT

Without an organized task list, users may forget deadlines or have difficulty identifying which tasks need attention first. TaskFlow addresses this problem by displaying task details, status, priority, and due dates in one interface.

## 1.2 PROJECT OBJECTIVE

The objective of this project is to develop a responsive task management application using HTML, CSS, and JavaScript. The application enables users to create, view, edit, delete, search, sort, filter, and track tasks through a clear interface.

2. TECHNOLOGY STACK

HTML5 is used to structure the page, form, dashboard, and task list.

CSS3 is used to style the interface and support responsive layouts.

JavaScript is used to implement task management and interactive features.

Browser Local Storage is used to store tasks in the current browser.

3. APPLICATION FEATURES

## 3.1 TASK MANAGEMENT

Users can create tasks with a title, description, priority, and due date. Existing tasks can be edited or deleted. The application asks for confirmation before deleting a task.

## 3.2 COMPLETION STATUS

Tasks can be marked as complete or pending. The status is displayed on each task card.

## 3.3 DASHBOARD

The dashboard displays the total number of tasks, pending tasks, completed tasks, and overdue tasks.

## 3.4 SEARCH, FILTERS, AND SORTING

Users can search tasks by title or description, filter them by completion status and priority, and sort them by due date, priority, or creation time.

3.5 OVERDUE INDICATOR

A pending task with a due date earlier than the current date is counted as overdue and receives a visible label and card highlight. Completed tasks are not counted as overdue.

3.6 DATA PERSISTENCE

Task data is stored in the browser's Local Storage. It remains available after refreshing the page in the same browser and on the same device.

4. APPLICATION WORKFLOW
5. The user opens the application in a web browser.
6. The user enters task details in the task form.
7. JavaScript validates the required fields and creates the task.
8. The task appears in the task list.
9. The user can search, filter, sort, edit, complete, or delete tasks.
10. Dashboard counts update when the task list changes.
11. Tasks are saved in Local Storage and restored when the page loads again.
12. USER INTERFACE AND IMPLEMENTATION

The interface is organized into a header, dashboard, task form, and task list.

The dashboard summarizes task progress. The form collects the information needed for a task. Search and filter controls help users locate tasks. Task cards display the title, description, priority, due date, status, and available actions.

CSS styles the page and provides layouts for different screen widths. JavaScript creates task cards, handles user actions, updates dashboard statistics, and saves data in Local Storage.

5.1 LOCAL STORAGE AND DATA HANDLING

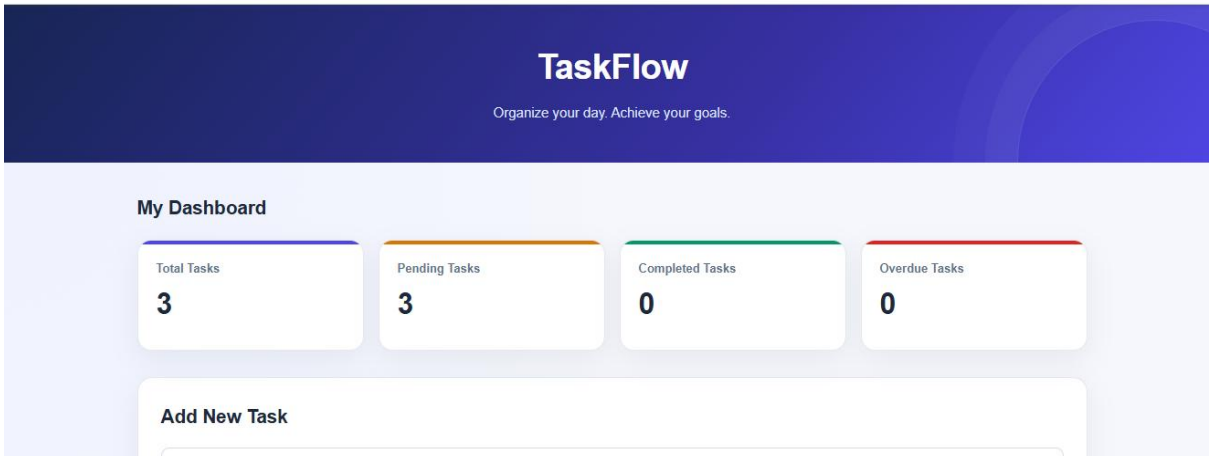
TaskFlow converts the task list into JSON text before saving it in Local Storage. When the application opens, it reads the saved text and converts it back into JavaScript data.

Local Storage belongs to the browser and device being used. Tasks are not automatically synchronized to an online account or another device. Clearing browser data may remove saved tasks.

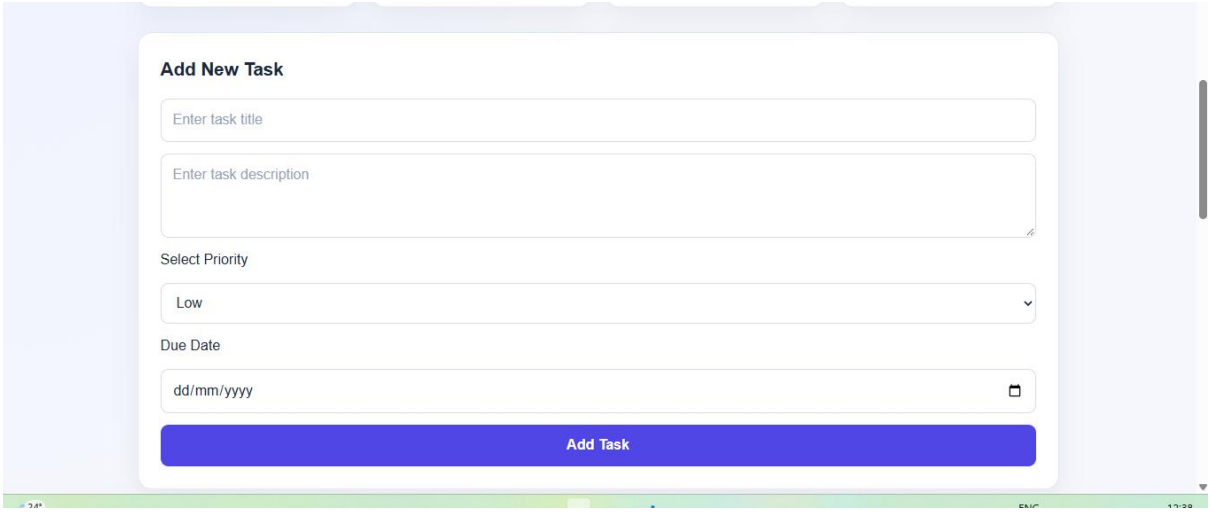
6.SCREENSHOTS

Insert the completed application screenshots at the indicated locations using the picture insertion option in your document editor.

6.1 MAIN DASHBOARD




6.2 TASK CREATION FORM




6.4 Mobile View

# TaskFlow

Organize your day. Achieve your goals.

## My Dashboard

Total Tasks
3

Pending Tasks
3

Completed Tasks
0

Overdue Tasks
0

## Add New Task

Enter task title

Enter task description

Select Priority

Low

Due Date

dd/mm/yyyy

Add Task

## My Tasks

Search tasks...

All Tasks

All Priorities

Newest First

### Organize Project Files

Check the TaskFlow files and arrange the project folders.

**Priority: Low**

Due Date: 2026-11-06

Status: Pending

Complete Edit Delete

### Prepare JavaScript Notes

Review functions, arrays, and event listeners.

**Priority: Medium**

Due Date: 2026-10-28

Status: Pending

Complete Edit Delete

### English Assignment

Finish the chapter questions and submit the assignment.

**Priority: High**

Due Date: 2026-10-25

Status: Pending

Complete Edit Delete

7. TESTING

Add a task with valid details
Expected result: The task appears in the list and the dashboard total updates.
Result: Pass

Submit with missing required details
Expected result: The incomplete task is not added.
Result: Pass

Edit a task
Expected result: The updated details appear in the task list.
Result: Pass

Delete a task
Expected result: Confirmation is requested before deletion.
Result: Pass

Complete or reopen a task
Expected result: The task status and dashboard totals update.
Result: Pass

Search for a task
Expected result: Matching tasks appear.
Result: Pass

Filter by status or priority
Expected result: Only matching tasks appear.
Result: Pass

Sort the task list
Expected result: Tasks appear in the selected order.
Result: Pass

Refresh the page
Expected result: Previously saved tasks load again.
Result: Pass

Check an overdue pending task
Expected result: It is counted and visibly marked as overdue.
Result: Pass

View on a mobile-sized screen
Expected result: The layout fits the narrow screen.
Result: Pass

8. CHALLENGES AND SOLUTIONS

### 8.1 RETAINING TASKS AFTER REFRESH

Challenge: Tasks should remain available after the browser page is refreshed.

Solution: Browser Local Storage is used to save and reload the task list.

### 8.2 IDENTIFYING OVERDUE TASKS

Challenge: Users need to recognize pending tasks whose due dates have passed.

Solution: The application compares each pending task's due date with the current date and displays an overdue count and visual indicator.

### 8.3 ORGANIZING MULTIPLE TASKS

Challenge: A growing task list can be harder to review.

Solution: Search, filters, sorting, priorities, and status labels help users find and organize tasks.

## 9. CONCLUSION

TaskFlow provides a simple way to manage tasks through a browser. It combines task creation and editing with priorities, due dates, completion status, search, filters, sorting, dashboard statistics, and Local Storage persistence.

The project provided practical experience with HTML, CSS, JavaScript, responsive design, browser storage, and user interface development.

## 10. FUTURE ENHANCEMENTS

Possible future improvements include user accounts, cloud-based task synchronization, reminders and notifications, categories and tags, a calendar view, recurring tasks, and integration with a backend API and database.

## REFERENCES

MDN Web Docs: HTML, CSS, and JavaScript documentation.

MDN Web Docs: Web Storage API documentation.

## PROJECT DETAILS

Project: TaskFlow: Task Management Web Application
Student: Nikhat Siddekee
Course: B.Tech in Computer Science and Engineering
Project Guide: Pratham Saxena